



Lesson 10: Convolutional Networks and Course Synthesis

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi
dell'Aquila

Agenda

- A convolution, and the two assumptions inside it
- Equivariance and pooling: how position stops mattering
- One defect, moved half an image
- Transfer learning, and its narrow window
- What ten lessons add up to

Exercise 9, before we start

- What counts is the **spread** reported beside each number, not the number
- The gap to watch for: a hand-written backward pass, never gradient-checked
- Conclusions drawn from a single seed, presented as differences
- Today the same obligation lands on an **architecture** claim

The sentence lesson 9 ended on

- “Permute the pixels consistently and nothing here notices”
- True of every model in that lesson, and an admission
- A 24×24 image is **not** 576 unrelated numbers
- Neighbouring pixels are the whole point of a photograph
- Today: what you get by refusing to throw the arrangement away

One assumption changes

- Same loss, same optimiser, same backpropagation as last week
- One new layer type, and inside it **two** assumptions
- Everything else today is consequence, or measurement
- No new mathematics after the first twenty-five minutes

Not “convolution wins on images”

- A random forest on raw pixels will come within **0.002** today
- Lesson 9’s dense network will lose to plain logistic regression
- Four hand-written numbers put every classical family at the ceiling
- What wins is the **representation**, not the model family

Three results that contradict a slogan

- Permuting the pixels costs the convolution **nothing** on detection
- Transfer learning helps in a **window**, and is negative outside it
- Freezing a pre-trained base hurt even the **good** source
- All three are measured this afternoon, in front of you

Five numbers so far; today the sixth

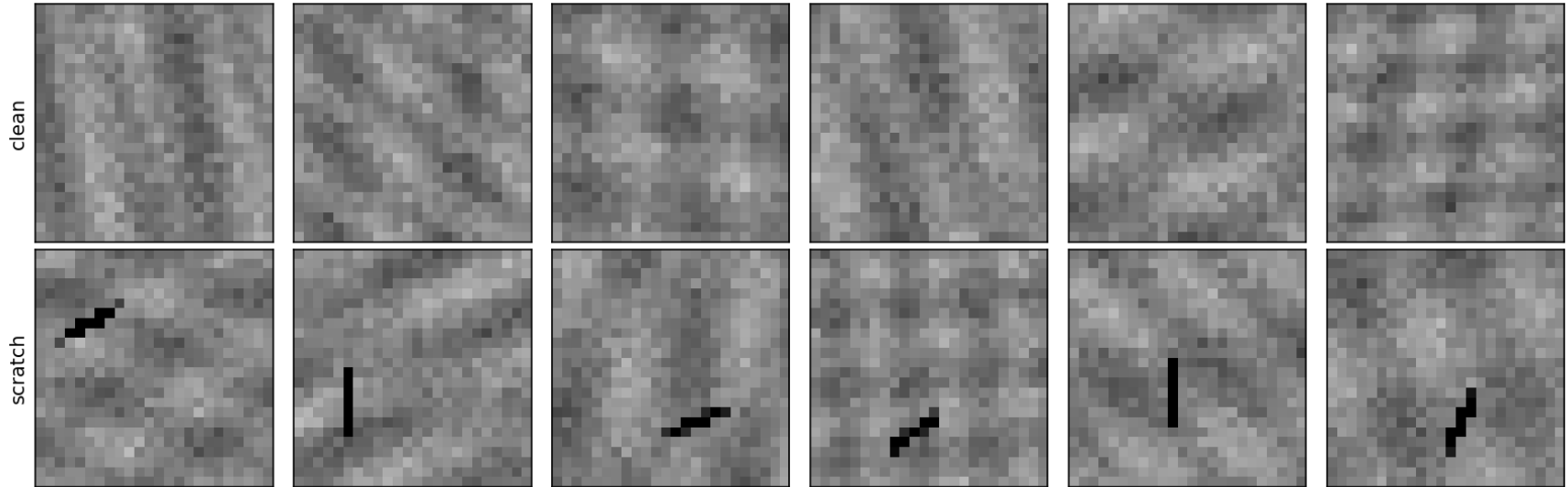
- **77%** accuracy on coin-flip labels, from leakage alone
- **94 of 128** imputed rows borrowed from the test set
- **365**: the largest coefficient a 0.01 penalty leaves
- **37 of 40** disguised accounts caught by reconstruction error
- A sigmoid layer divides the gradient by about **four**

Meridian's wafers

- Lesson 9's optical sensors are cut from silicon wafers
- Every die is photographed at **24×24 pixels** and graded
- Rejected if it carries a defect: a scratch of about seven pixels, or a bright particle
- Film thickness varies smoothly across each die

Six clean dies, six scratched

Meridian's wafer inspection: six clean dies, six with a scratch



Two properties decide everything

- **Local:** a handful of the 576 pixels matter, the rest is irrelevant
- **Position-independent:** top left and bottom right are the same event
- Both were visible in the twelve photographs
- Every design choice this afternoon follows from these two lines

The grading station is wrong 2% of the time

- Published error rate **2%**; notebook 01's batch drew **2.55%**
- A high draw at 2.5 standard deviations, not a bug
- Every score today is read against the ceiling of the set it was measured on
- For the 2,000-image test set that ceiling is **0.9800**

One neuron per pixel is the wrong shape

- A dense unit on this image has **576 weights**, one per pixel
- A unit that has learned to spot a scratch here...
- ...knows nothing about the same scratch three pixels to the left
- Those are different weights, trained by different examples
- Nothing in the architecture connects them

80 weights against 147,712

layer	parameters
dense, 256 units	147,712
dense, 1024 units	590,848
convolutional, 8 kernels of 3×3	80
convolutional, 32 kernels of 5×5	832

Compression is the least interesting reading

- The convolutional layer produces **more** numbers, not fewer
- Eight feature maps of 24×24 is **4,608** outputs, against 256
- It is not a smaller layer: it is a differently **tied** one
- The 1,846-fold parameter ratio is a side effect

What weight sharing buys is evidence

- The same nine weights are applied at **every** position
- So one scratch, anywhere, teaches the detector **everywhere**
- The dense layer must be taught position by position
- Not a memory saving: a **data** saving
- We price it, in images, at 1:35

Before the mathematics: what would you compute?

- Look at the twelve dies again
- What number would you compute from an image to grade it?
- If your answer involves a small window and the word “anywhere”...
- ...you have already derived the next forty minutes

The convolution, in words

- Take a small array of weights: the **kernel**
- Slide it over the image, one position at a time
- At each position, write down the weighted sum of the pixels it covers
- The result is a **feature map**: one number per position
- The kernel is small; the image is not

One weighted sum, at every position

$$(I * K)[r, c] = \sum_{i=0}^{f-1} \sum_{j=0}^{f-1} I[r+i, c+j] K[i, j]$$

Strictly, this is cross-correlation

- A true convolution flips the kernel before multiplying
- Every deep learning library computes the formula above and calls it convolution
- The kernel is **learned**, so the flip changes nothing that matters
- Notebook 01 agrees with scipy to 8.9×10^{-16} : rounding, not error

How big is the output?

$$\text{output size} = \left\lfloor \frac{n + 2p - f}{s} \right\rfloor + 1$$

Worked on this lesson's 24×24 images

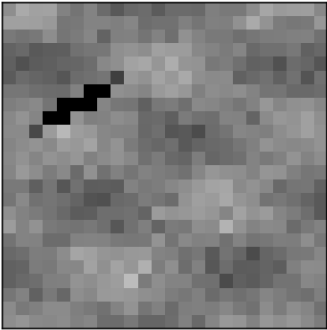
kernel	padding	stride	output
3	0	1	22
3	1	1	24
5	2	1	24
3	1	2	12

Two cases cover almost everything

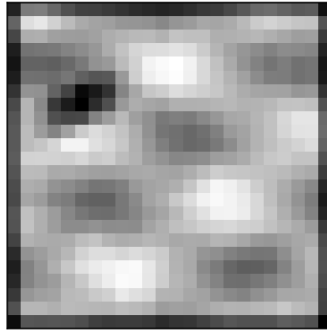
- **3×3 with padding 1 leaves the size unchanged:** Keras calls this `padding="same"`
- In general, $\text{padding} = (\text{kernel} - 1) / 2$ for odd kernels
- **Stride 2 halves it:** the cheap alternative to pooling
- Almost every architecture you will read is built from these two

Four kernels written down, not learned

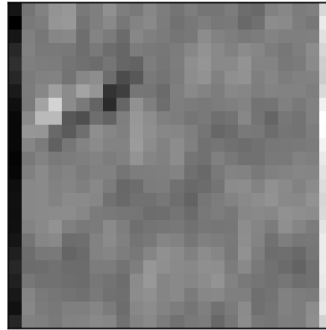
the die



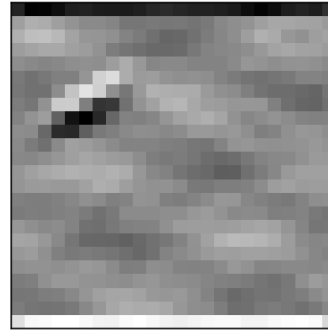
blur (local average)



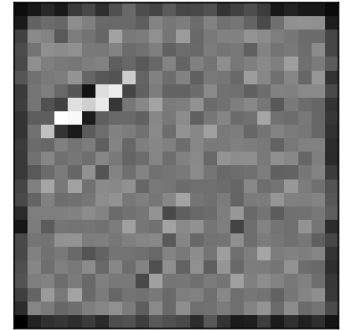
vertical edge



horizontal edge



darker than neighbours



A constant patch gives exactly zero

$$\sum_{i,j} v \cdot K[i,j] = v \sum_{i,j} K[i,j] = v \cdot 0 = 0$$

Nine numbers, no training: 3.129 against 1.118

- Centre +8, each of the eight neighbours −1, sum exactly zero
- Blind to brightness, sensitive only to **local contrast**
- Strongest response on a defective die: **3.129**
- Strongest response on a clean die: **1.118**
- Nothing has been trained yet

Notebook 1, live

- The convolution in eleven lines, checked against scipy
- Output sizes measured, then compared with the formula
- Four kernels by hand, and the zero-sum one that works
- Equivariance checked at four offsets, then pooling

Break

- Twelve minutes

Shift the input, and the response shifts with it

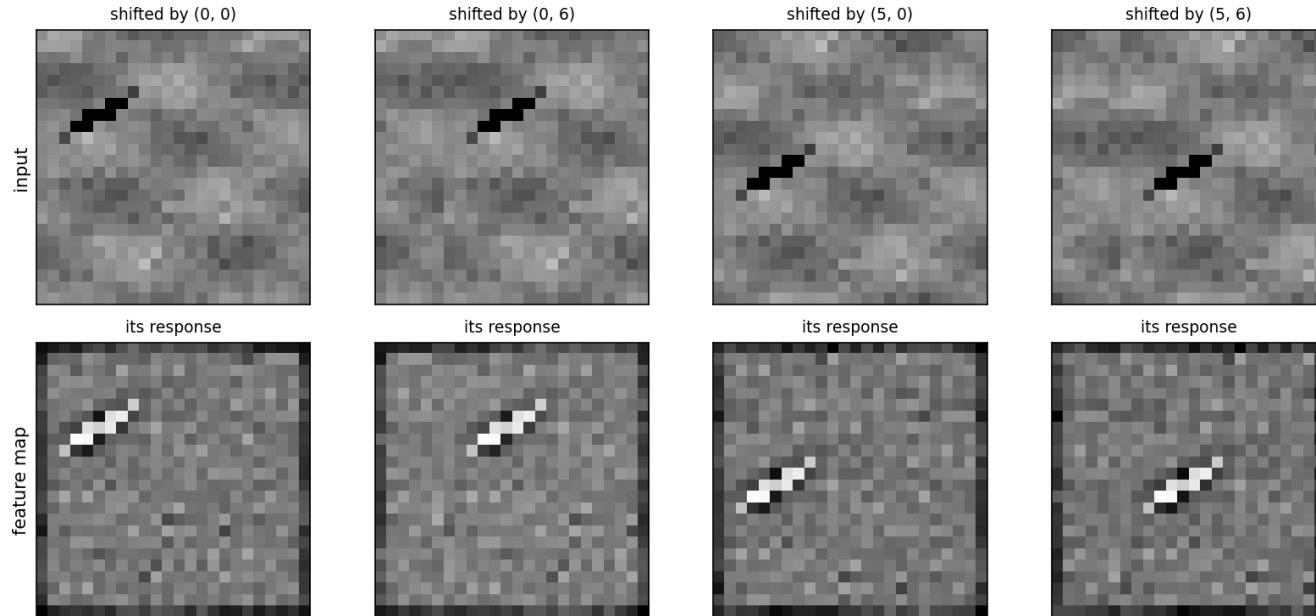
- **Equivariance:** the response does not change, it **moves**
- It follows in one line from the definition
- Notebook 01 checks it at four offsets: largest difference exactly **0**
- A dense layer has nothing of the kind
- Shift by one pixel and all 576 weights meet different pixels

Shift, then convolve, is convolve, then
shift

$$(\text{shift}_d I) * K = \text{shift}_d(I * K)$$

The bright spot tracks the defect

move the defect and the bright spot moves with it, the detector does not have to be retrained

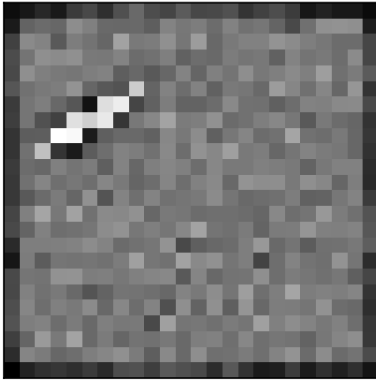


Pooling: from *where* to *whether*

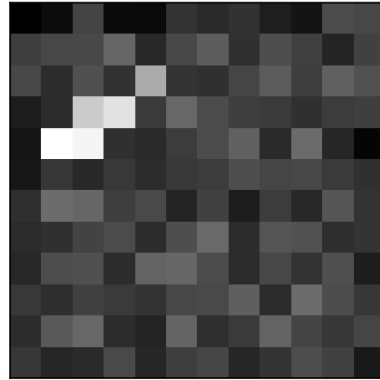
- Equivariance says the response **moves** with the defect
- To grade a die we do not want it to move
- We want one number: did a strong response occur **anywhere?**
- Max pooling takes the maximum over a window, discarding the position
- 24×24 down to 3×3 , and the maximum is still 3.129

Pooled three times; the maximum never moves

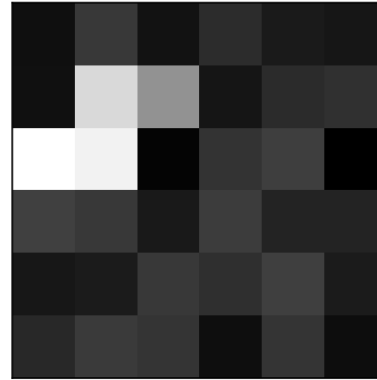
feature map
24x24



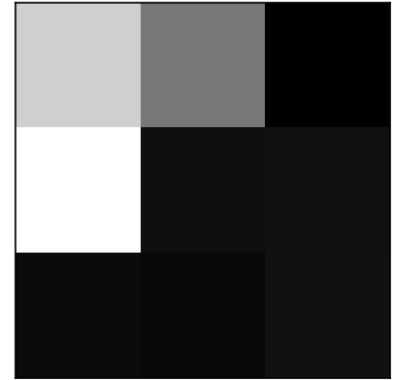
after 1 pooling step
12x12



after 2 pooling steps
6x6



after 3 pooling steps
3x3



Three things pooling does

- Turns equivariance into **invariance**: one pixel of shift often changes nothing
- Enlarges the receptive field: 3×3 after pooling covers 6×6 of the image
- Discards spatial precision, which is a **cost** when *where* is the answer
- **Global** max pooling: one number per kernel, over the whole map

1,537 parameters, in full

layer	output shape	parameters
conv, 8 kernels of 3×3 , same	$24 \times 24 \times 8$	80
max pool 2×2	$12 \times 12 \times 8$	0
conv, 16 kernels of 3×3 , same	$12 \times 12 \times 16$	1,168
max pool 2×2	$6 \times 6 \times 16$	0
global max pool	16	0
dense 16, then dense 1	1	289
total		1,537

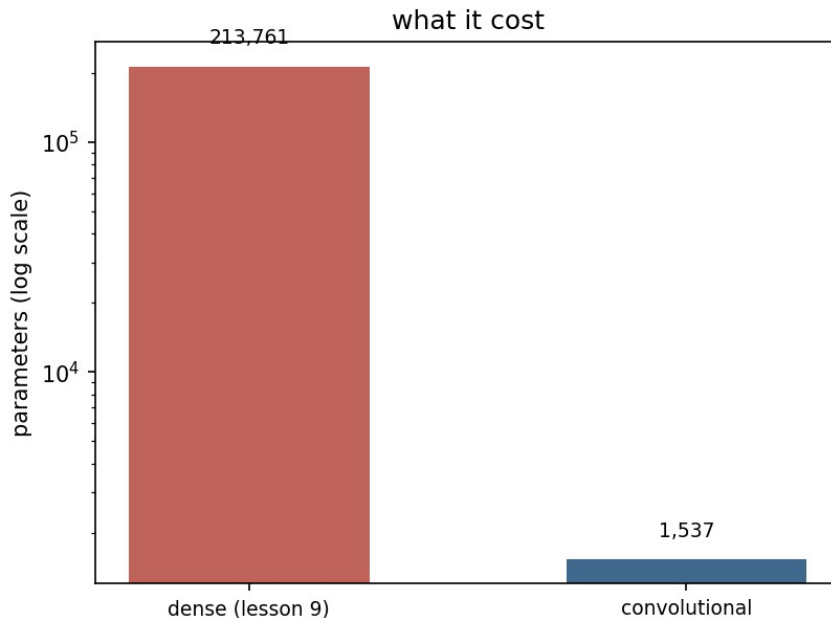
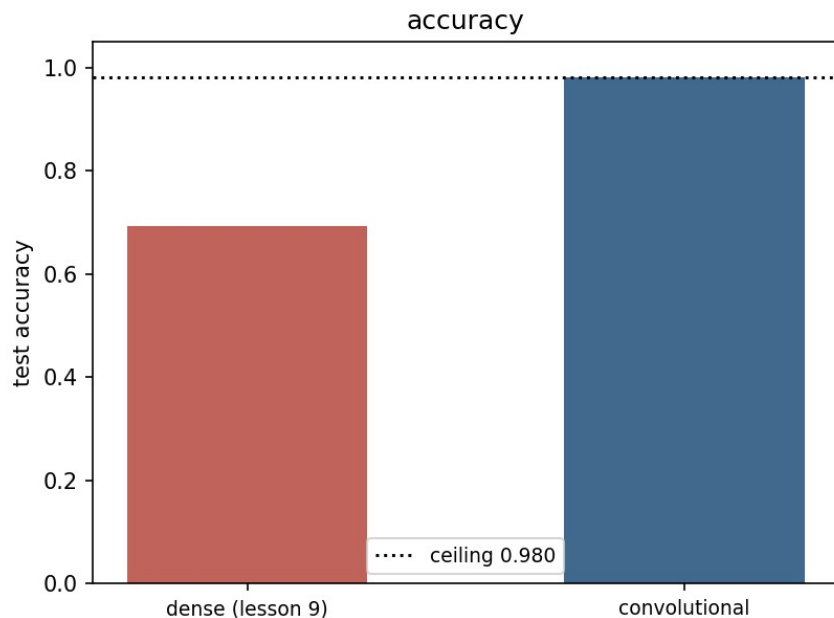
The count students get wrong: 1,168

- Each of the 16 kernels is 3×3 **across all eight input channels**
- So each one holds $9 \times 8 = 72$ weights, not 9
- $16 \times 72 + 16 = 1,168$
- A kernel's depth always matches the input's channel count
- Only its two spatial dimensions are yours to choose

Head to head, and where the missing 2% lives

- Dense network, lesson 9: **0.6928**, standard deviation 0.0342
- Convolutional: **0.9800**, standard deviation 0.0000
- Against the **true** grade the convolutional network scores **1.0000**
- Its visible 0.98 is the grading station's error and nothing else
- There is no gap left for a better architecture to close

1,537 parameters against 213,761



A defect where none has been seen

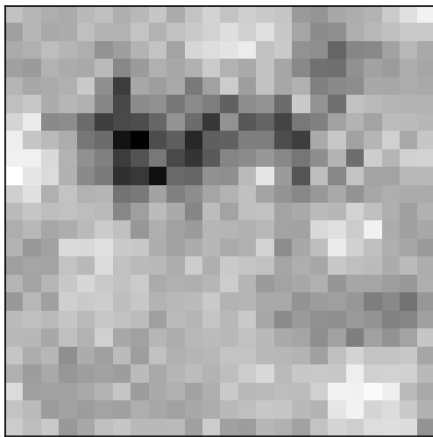
- Training defects live only in the **top** band of rows
- One test set keeps them there; the other moves them to the **bottom**
- Same generator, same defect, same contrast, same number of images
- Only the row changed
- Predict the two numbers before you see them

The number to carry out of the room

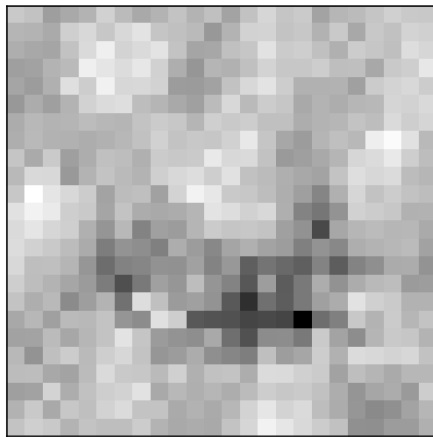
network	trained band	moved to the other half	cost
dense (lesson 9)	0.8567	0.4617	-0.3950
convolutional	0.9800	0.9847	+0.0047

Where the defects were, and what it cost

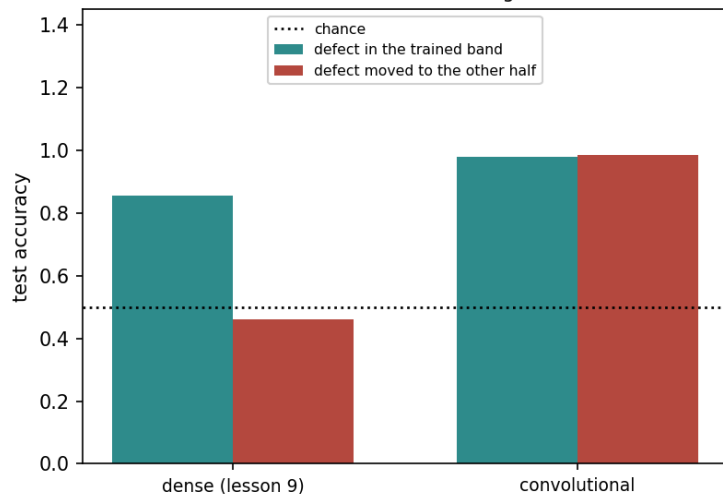
trained here: rows 5-11



tested here: rows 13-19



the same defect, half an image lower



Below chance, and why nothing degraded

- Most students predict a drop that stops well above chance
- The reasoning is sound: shifts usually degrade, they do not vanish
- But this is not a change of **degree** in the inputs
- It is a change of **which** inputs carry the signal
- Knowledge stored per position, at positions never trained

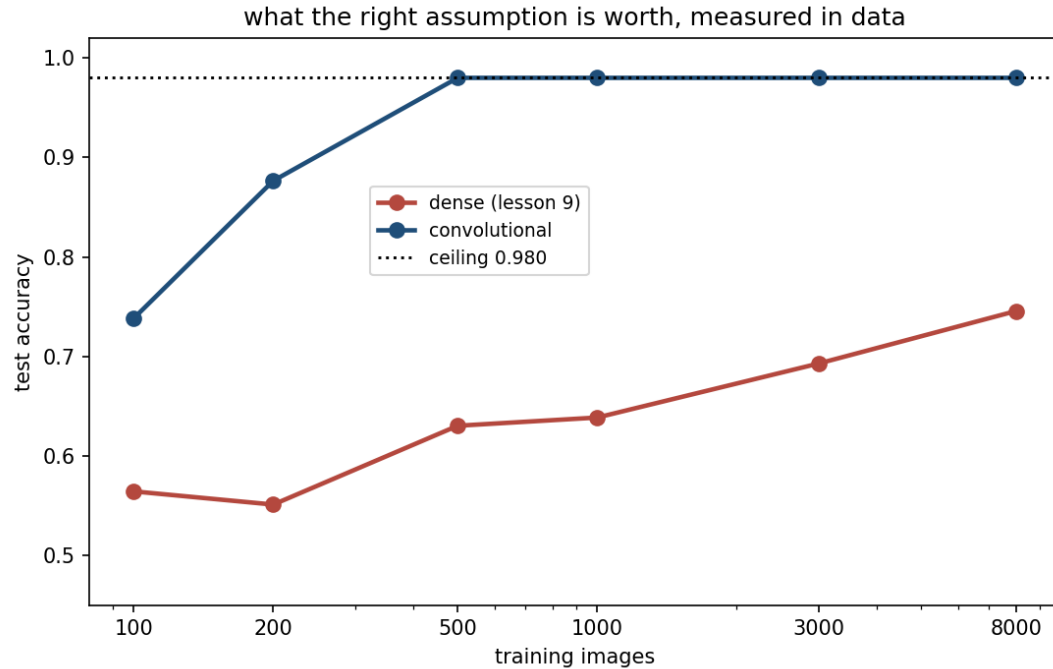
A convolutional layer is a restricted dense layer

- Every function it computes, a dense layer can compute too
- Tie the shared entries, zero the rest: it is strictly **less** expressive
- The functions it gives up all treat row 5 differently from row 15
- Nobody wanted any of those
- **A model that cannot express a wrong answer need not learn to avoid it**

Augmentation buys 8 points and leaves it 44 short

- Show the dense network shifted copies: a moved scratch is still a scratch
- Dense: $0.4617 \rightarrow \mathbf{0.5423}$, a real gain of $\mathbf{+0.0807}$
- Convolutional: $0.9847 \rightarrow 0.9847$, unchanged, with nothing to gain
- Augmentation **teaches** an invariance: approximate, and forgettable
- Architecture **asserts** it: exact, free, permanent

At the ceiling by 500 images; 0.235 short at 8,000



Notebook 2, live

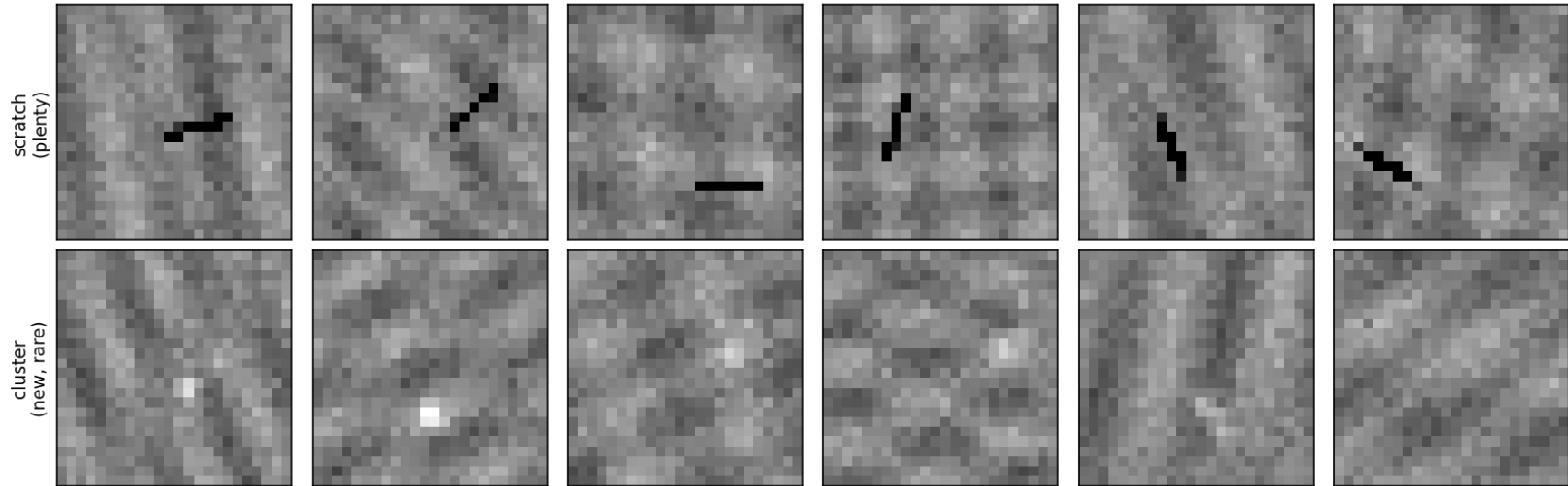
- Both architectures trained head to head, with seed spreads
- The translation experiment, run in front of you
- Augmentation with the epoch budget equalised
- The learning curve from 100 images to 8,000
- The permutation test: hold your prediction

A new defect appears on a Monday

- A contamination **cluster**: several faint specks together
- By Friday, a few dozen labelled photographs
- Section 8 says the network needs several hundred
- Waiting six months is no answer: the line ships now
- Can we borrow from a network trained on something else?

The defect the line knows, and Monday's

the defect the line knows, and the one that appeared on Monday



Start from someone else's weights

- **Transfer learning**: initialise from a network trained on a related task
- The argument: early layers detect local contrast, which is not scratch-specific
- Two knobs: which **source** task, and which layers stay **frozen**
- Both knobs matter more than the received wisdom suggests

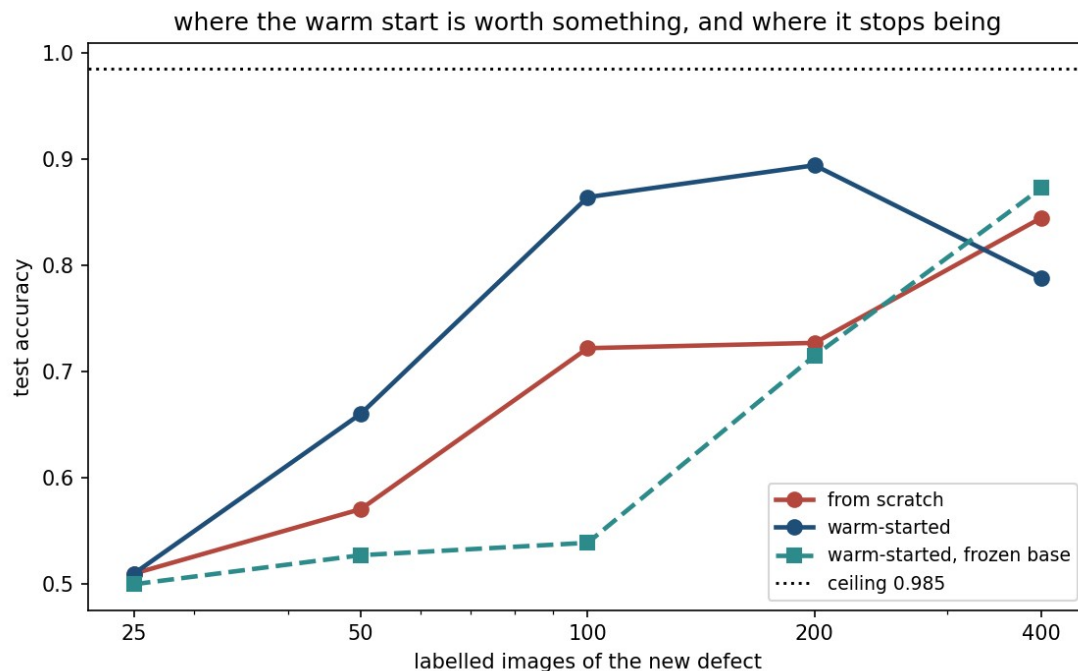
The source task is the whole game

- Pre-train on a task **nobody at Meridian wants solved**
- Naming which of three types a die carries: clean, scratch, particle
- Grading pass/fail teaches **one** detector; typing forces description
- **0.9993** on the source task: a receipt, not a result
- Transfer carries what the source **forced** it to learn

A window, not a slope

labelled images	from scratch	warm-started	gain
25	0.5100	0.5100	0.0000
100	0.7220	0.8640	+0.1420
200	0.7270	0.8940	+0.1670
400	0.8447	0.7877	-0.0570

The window, drawn



When transfer makes things worse

labelled images	from scratch	frozen, typing source	frozen, scratches only
50	0.5707	0.5273	0.5150
100	0.7220	0.5390	0.5047
200	0.7270	0.7150	0.5350

Two rules, and neither is “always pre-train”

- Transfer carries what the source task **forced** the network to learn
- Ask what the source required before trusting its features
- Freezing is a **bet** that the borrowed features are already right
- **In doubt: warm start, and leave everything trainable**

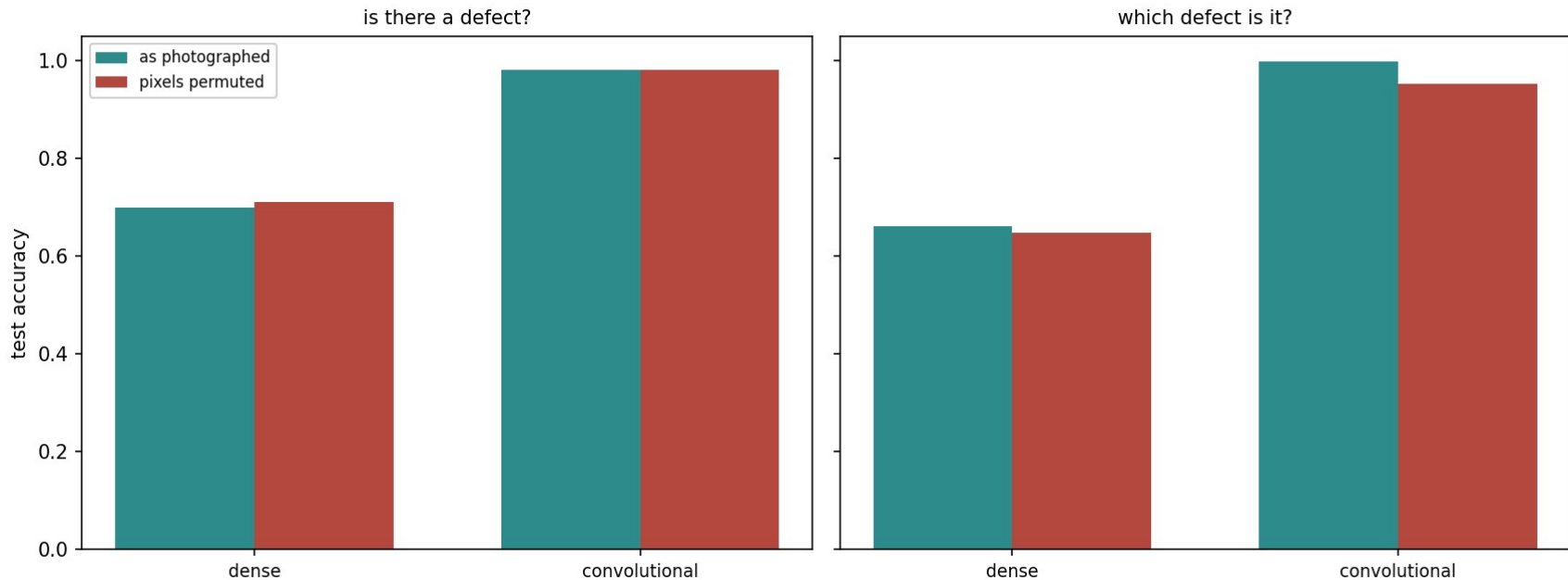
Notebook 3, live

- Pre-training on the task nobody wants solved, up to 0.9993
- The transfer window, measured at five dataset sizes
- Negative transfer, from a source chosen badly on purpose
- Every family the course has met, on one problem

What is the convolution actually using?

- “Images have spatial structure” is **two** claims, not one
- Permute every image’s pixels identically, then train
- Detection: **0.9798 → 0.9800**, no cost at all
- Typing: **0.9970 → 0.9523**, 4.5 points
- The dense network notices neither

Nothing on detection, 4.5 points on typing

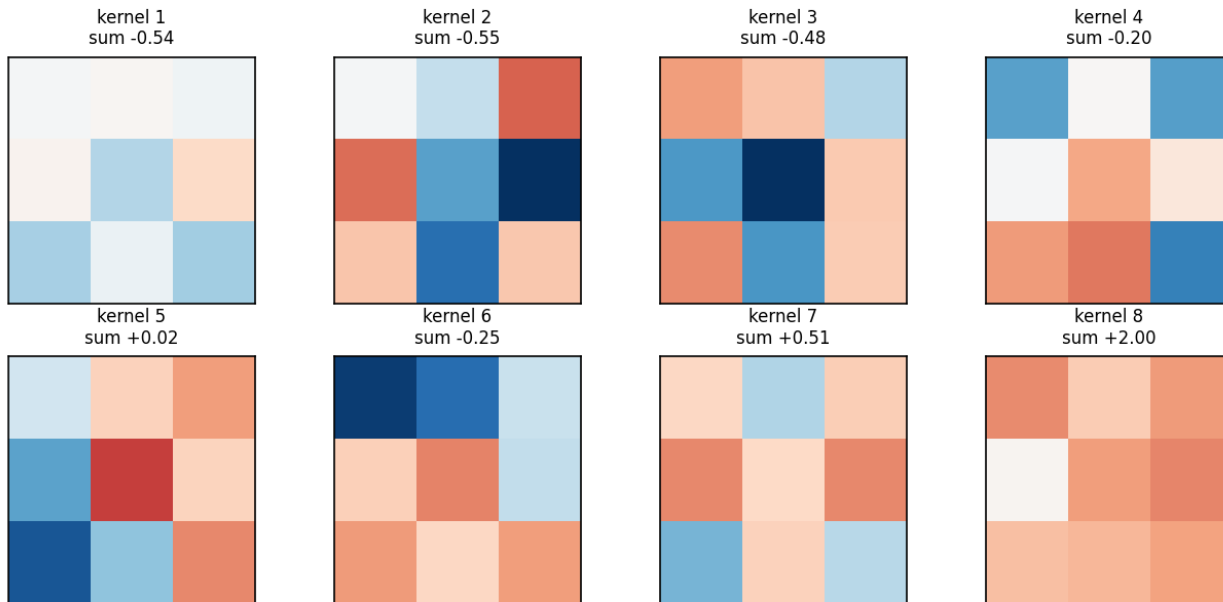


Two assumptions, and only one is about arrangement

assumption	what it says	needed by
weight sharing	a pattern means the same thing wherever it appears	both tasks
locality	nearby pixels belong together	only the typing task

What the first layer learned, unprompted

the eight 3×3 kernels the first layer learned, red positive, blue negative



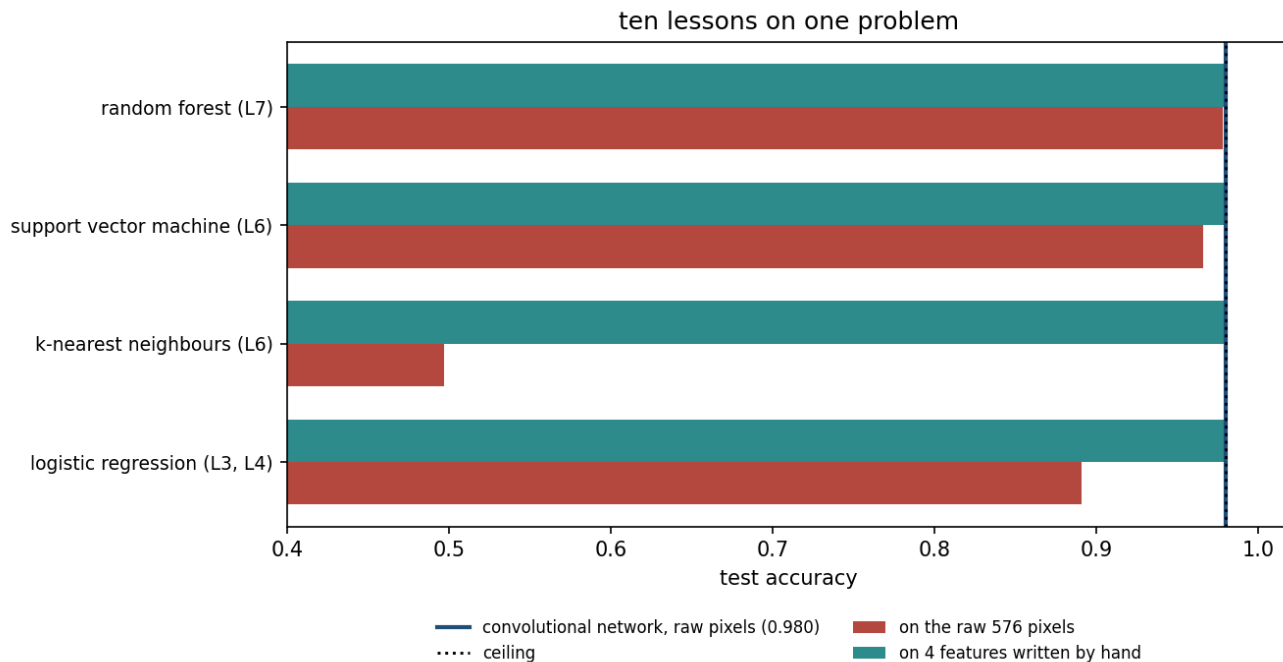
What this lesson did not need

- Today ran on a laptop processor, no graphics card
- Deeper networks revive lesson 9's gradient problem
- **Batch normalisation** rescales each layer's activations
- **Residual connections** let the gradient skip a layer
- Neither is needed at 24×24

Ten lessons on one problem

family	raw pixels	4 hand-made features
logistic regression (3, 4)	0.8905	0.9800
k-nearest neighbours (6)	0.4970	0.9800
support vector machine (6)	0.9660	0.9800
random forest (7)	0.9780	0.9795
dense network (9)	0.7430	-
convolutional network (10)	0.9800	-

Four hand-written numbers reach the ceiling



The representation matters more than the model

- A model turns a **representation** into a decision
- Classical methods make you build it; networks learn it, paid for in data
- Lesson 9: a hidden layer worth 39 points on one problem, 0.09 on another
- Today: a random forest within **0.002** of the convolutional network
- **Neither route is a default**

Homework: the last one

- **Exercise 10**, no lesson left to discuss it: work it for the exam
- `Exercises/10_convolutional_networks.md`
- The one you will have had **least practice defending**
- Write the reasoning down as you go, not afterwards

What ten weeks add up to

- **0.8567 to 0.4617** for a dense network, on a defect moved half a die
- The convolutional one went 0.9800 to 0.9847, because it cannot ask *where*
- An assumption that is true beats flexibility that is not
- The representation is doing most of the work; the only question is who builds it
- Thank you, and good luck with the project